

Credit Risk Prediction and Explainable Credit Scoring System

Author: Megha Malhotra

Objective

This project develops a machine learning based credit risk assessment framework to identify customers likely to become seriously delinquent within the next two years. The solution follows an end-to-end analytics workflow commonly used by credit bureaus, banks, and risk analytics firms such as TransUnion.

Business Problem

- Financial institutions face significant losses due to loan defaults. The key challenge is: Can we identify high-risk customers before extending credit?
- Early identification of risky borrowers enables:
- Better lending decisions
 - Lower default rates
 - Improved risk-adjusted profitability
 - More accurate credit scoring

Success Criteria

- Develop a predictive model that:
- Identifies customers likely to default
- Achieves strong ROC-AUC performance
- Provides interpretable business explanations
- Supports future deployment into a credit risk workflow

Import Dataset

Dataset: Give Me Some Credit Target Variable:

SeriousDlqin2yrs

1 = Customer defaulted

0 = Customer did not default

```
1 !wget https://drive.google.com/uc?id=1ix-ezN3RLNmaWf1zhTup2hB5sLkzDAq
Download...
From: https://drive.google.com/uc?id=1ix-ezN3RLNmaWf1zhTup2hB5sLkzDAq
To: /content/cs-training.csv
100% 7.56M/7.56M [00:00<00:00, 14.2MB/s]

1 import pandas as pd
2
3 df=pd.read_csv('cs-training.csv')
4 df.head()
```

	Unnamed: 0	SeriousDlqin2yrs	RevolvingUtilizationOfUnsecuredLines	age	NumberOfTime30-59DaysPastDueNotWorse	DebtRatio	MonthlyIncome	NumberOfOpenCreditLinesAndLoans	NumberOfTimes90DaysLate	NumberRealEstateLoansOrLines	NumberOfTime60-89DaysPastDueNotWorse	NumberOfDependents
0	1	1	0.766127	45	2	0.802982	9120.0	13	0	6	0	2.0
1	2	0	0.957151	40	0	0.121876	2600.0	4	0	0	0	1.0
2	3	0	0.658180	38	1	0.085113	3042.0	2	1	0	0	0.0
3	4	0	0.233810	30	0	0.036050	3300.0	5	0	0	0	0.0
4	5	0	0.907239	49	1	0.024926	63568.0	7	0	1	0	0.0

Import Libraries

- The project uses:
- Pandas for data manipulation
 - NumPy for numerical calculations
 - Matplotlib and Seaborn for visualisation
 - Scikit-Learn for machine learning
 - XGBoost for advanced risk modelling
 - SHAP for explainable AI

```
1 !pip install xgboost shap -q

1 import pandas as pd
2 import numpy as np
3
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.model_selection import train_test_split
8 from sklearn.preprocessing import StandardScaler
9 from sklearn.impute import SimpleImputer
10 from sklearn.metrics import (
11     classification_report,
12     confusion_matrix,
13     roc_auc_score,
14     RocCurveDisplay
15 )
16
17 from sklearn.linear_model import LogisticRegression
18 from sklearn.ensemble import RandomForestClassifier
19
20 import warnings
21 warnings.filterwarnings("ignore")
```

▼ Data Understanding

Before modelling, it is important to understand

- Dataset dimensions
- Data types
- Missing Values
- Variable distributions

```
1 df.shape

(150000, 12)
```

```
1 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150000 entries, 0 to 149999
Data columns (total 12 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Unnamed: 0                            150000 non-null int64
 1   SeriousDlqin2yrs                     150000 non-null int64
 2   RevolvingUtilizationOfUnsecuredLines 150000 non-null float64
 3   age                                   150000 non-null int64
 4   NumberOfTime30-59DaysPastDueNotWorse 150000 non-null int64
 5   DebtRatio                             150000 non-null float64
 6   MonthlyIncome                         120269 non-null float64
 7   NumberOfOpenCreditLinesAndLoans      150000 non-null int64
 8   NumberOfTimes90DaysLate               150000 non-null int64
 9   NumberRealEstateLoansOrLines          150000 non-null int64
10   NumberOfTimes60-89DaysPastDueNotWorse 150000 non-null int64
11   NumberOfDependents                   146076 non-null float64
dtypes: float64(4), int64(8)
memory usage: 13.7 MB
```

1 df.describe()												
Unnamed: 0 SeriousDlqin2yrs RevolvingUtilizationOfUnsecuredLines age NumberOfTime30-59DaysPastDueNotWorse DebtRatio MonthlyIncome NumberOfOpenCreditLinesAndLoans NumberOfTimes90DaysLate NumberRealEstateLoansOrLines NumberOfTime60-89DaysPastDueNotWorse NumberOfDependents												
count	150000.000000	150000.000000	150000.000000	150000.000000	150000.000000	150000.000000	1.202690e+05	150000.000000	150000.000000	150000.000000	150000.000000	146076.000000
mean	75000.500000	0.066840	6.048438	52.295207	0.421033	353.005076	6.670221e+03	8.452760	0.265973	1.018240	0.240387	0.757222
std	43301.414527	0.249746	249.755371	14.771866	4.192781	2037.818523	1.438467e+04	5.145951	4.169304	1.129771	4.155179	1.115086
min	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000
25%	37500.750000	0.000000	0.029867	41.000000	0.000000	0.175074	3.400000e+03	5.000000	0.000000	0.000000	0.000000	0.000000
50%	75000.500000	0.000000	0.154181	52.000000	0.000000	0.366508	5.400000e+03	8.000000	0.000000	1.000000	0.000000	0.000000
75%	112500.250000	0.000000	0.559046	63.000000	0.000000	0.868254	8.249000e+03	11.000000	0.000000	2.000000	0.000000	1.000000
max	150000.000000	1.000000	50708.000000	109.000000	98.000000	329664.000000	3.008750e+06	58.000000	98.000000	54.000000	98.000000	20.000000

▼ Initial Observations

Document findings here after reviewing outputs. Example:

- Dataset contains approximately 150k customers.
- Several variables contain missing values.
- Significant variations exist across financial metrics.
- Default appears to be minority class.

▼ Data Quality Assessment

Missing values can negatively impact model performance and must be addressed before training

```
1 df.isnull().sum()
```

	0
Unnamed: 0	0
SeriousDlqin2yrs	0
RevolvingUtilizationOfUnsecuredLines	0
age	0
NumberOfTime30-59DaysPastDueNotWorse	0
DebtRatio	0
MonthlyIncome	29731
NumberOfOpenCreditLinesAndLoans	0
NumberOfTimes90DaysLate	0
NumberRealEstateLoansOrLines	0
NumberOfTime60-89DaysPastDueNotWorse	0
NumberOfDependents	3924

dtype: int64

```
1 missing=(df.isnull().sum().sort_values(ascending=False))
```

Exploratory Data Analysis

The goal of EDA is to understand:

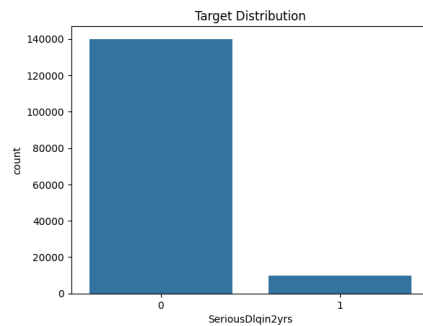
- Customer behaviour
- Relationship between variables
- Potential risk indicators

```
1 df['SeriousDlqin2yrs'].value_counts()
```

SeriousDlqin2yrs	count
0	139974
1	10026

dtype: int64

```
1 sns.countplot(x='SeriousDlqin2yrs', data=df)
2
3 plt.title("Target Distribution")
4 plt.show()
```



Business Interpretation: Class Imbalance

The dataset contains approximately:

- Non-default customers: 139,974
- Default customers: 10,026

This means only around 6.7% of borrowers defaulted.

Why This Matters

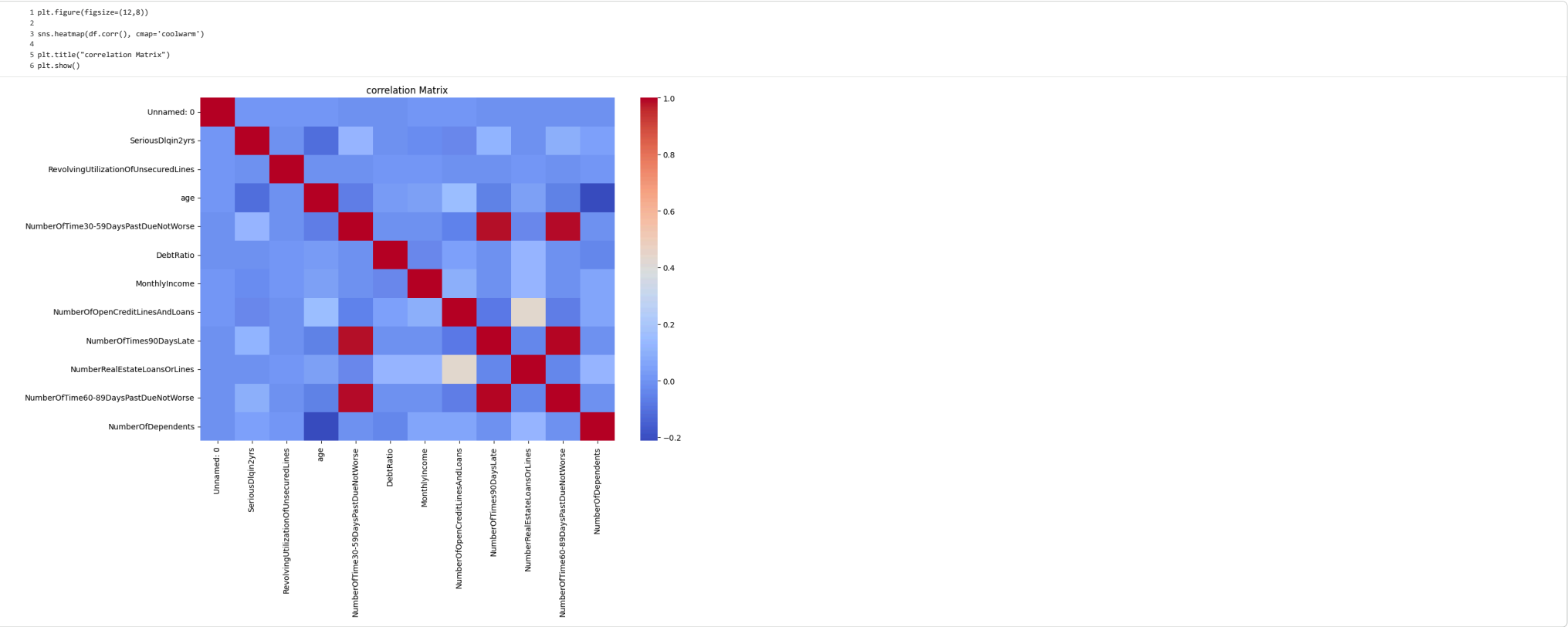
Class imbalance is common in lending data.

If a model predicts every customer as non-default, it would still achieve high accuracy but provide little business value.

For this reason, ROC-AUC is selected as the primary evaluation metric instead of accuracy.

Business Impact

The objective is not simply to maximize accuracy but to correctly identify risky borrowers before credit is approved.



Correlation Analysis

Key observations:

- Delinquency variables exhibit positive correlation with default behaviour.
- Several late-payment indicators appear closely related and may contain overlapping information.
- No severe multicollinearity issues are immediately visible.

Business Interpretation

Past payment behaviour is often one of the strongest predictors of future default.

This finding supports the hypothesis that customers with repeated payment delays present elevated credit risk.

Outlier Analysis

Financial datasets frequently contain extreme observations.

Identifying unusual values is important because they can distort model performance and statistical summaries.

Examples observed include:

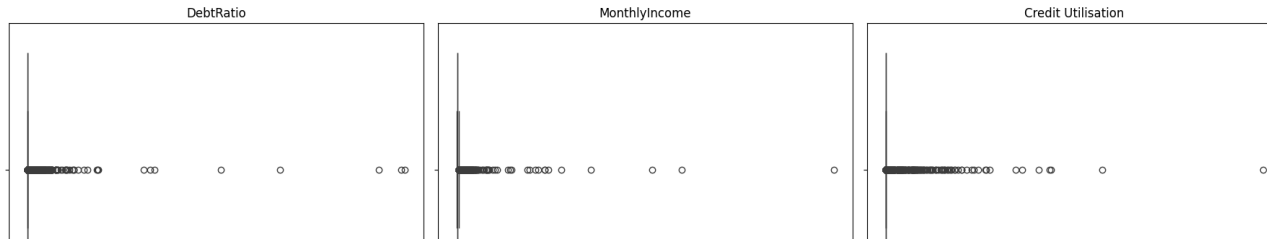
- Extremely high Debt Ratios
- Extremely high Income values
- Unusually high Credit Utilisation values



```

12 )
13 ax[2].set_title("Credit Utilisation")
14
15 plt.tight_layout()
16 plt.show()

```



Outlier Findings

Several variables contain extreme values.

Rather than removing these records, they are retained because unusual financial behaviour may itself be a signal of elevated credit risk.

Future model iterations may evaluate capping or winsorization techniques.

▼ Data Cleaning

Missing values will be imputed using median values.

The median is preferred because financial data often contains outliers.

```

1 df.drop(df.columns[0],axis=1,inplace=True)

```

```

1 df["MonthlyIncome"].fillna(df["MonthlyIncome"].median(),
2                               inplace=True)
3
4 df["NumberOfDependents"].fillna(df["NumberOfDependents"].median(),inplace=True)

```

```

1 df.isnull().sum()

```

```

 SeriousDlqin2yrs      0
RevolvingUtilizationOfUnsecuredLines  0
      age              0
NumberOfTime30-59DaysPastDueNotWorse  0
      DebtRatio         0
      MonthlyIncome     0
NumberOfOpenCreditLinesAndLoans      0
      NumberOfTimes90DaysLate  0
      NumberRealEstateLoansOrLines  0
NumberOfTime60-89DaysPastDueNotWorse  0
      NumberOfDependents      0
dtype: int64

```

▼ Feature Engineering

Feature engineering creates additional business-relevant variables that may improve prediction performance.

```

1 df["MonthlyDebt"]=(df["DebtRatio"]*df["MonthlyIncome"])

```

```

1 df["IncomePerDependent"]=(df["MonthlyIncome"]/(df["NumberOfDependents"]+1))

```

```

1 df["LatePaymentScore"]=(
2     df["NumberOfTime30-59DaysPastDueNotWorse"]
3     +df["NumberOfTimes90DaysLate"]
4     +df["NumberOfTime60-89DaysPastDueNotWorse"]
5 )

```

```

1 df["TotalPastDueEvents"] = (
2     df["NumberOfTime30-59DaysPastDueNotWorse"]
3     + df["NumberOfTime60-89DaysPastDueNotWorse"]
4     + df["NumberOfTimes90DaysLate"]
5 )

```

Total Past Due Events

This feature captures the cumulative number of delinquency incidents across all severity levels.

Business logic suggests that repeated delinquency behaviour increases future default risk.

Business Rationale

Late payment history is one of the stroingest indicatos of future delinquency.

The engineered LatePaymentScore captures cumulative payment behaviour.

▼

Data Preparation

Prepare training and testing datasets

```
1 X=df.drop("SeriousDlqin2yrs",
2         axis=1)
3
4 y=df["SeriousDlqin2yrs"]
5
6

1 X_train, X_test, y_train, y_test= train_test_split( X,y,
2                                             test_size=0.2, random_state=42, stratify=y)

1 scaler=StandardScaler()
2
3 X_train_scaled= scaler.fit_transform(X_train)
4
5 X_test_scaled=scaler.transform(X_test)
```

Data Splitting Strategy

The dataset was divided into:

- 80% Training Data
- 20% Testing Data

Stratified sampling was used to preserve the default rate across both datasets.

This approach ensures a realistic evaluation of model performance.

▼

Baseline Model: Logistic Regression

Logistic Regression is widely used in credit scoring because:

- Easy to explain
- Regulation-friendly
- Fast to Deploy

```
1 lr=LogisticRegression(max_iter=1000)
2
3 lr.fit(X_train_scaled, y_train)
4
5

▼  LogisticRegression ⓘ ⓘ
LogisticRegression(max_iter=1000)

1 lr_pred= lr.predict(X_test_scaled)
2
3 lr_prob=lr.predict_proba(X_test_scaled)[:1]
4
5

1 lr_prob = lr.predict_proba(X_test_scaled)[: , 1]
2 print(roc_auc_score(y_test,lr_prob))
3
4 print(classification_report(y_test,lr_pred))

0.7155679830607443
      precision    recall  f1-score   support

      0      0.94      1.00      0.97      27995
      1      0.57      0.04      0.08       2005

 accuracy      0.93      30000
 macro avg      0.75      0.52      0.52      30000
weighted avg      0.91      0.93      0.91      30000
```

Logistic Regression Interpretation

ROC-AUC: 0.716

The model demonstrates moderate predictive capability.

Advantages:

- Highly interpretable

- Regulatory friendly
- Commonly used in traditional credit scorecards

Limitations:

- May struggle to capture nonlinear risk patterns
- Lower predictive power than ensemble methods

Random Forest

Random Forest captures complex interactions between variables.

```
1 rf= RandomForestClassifier(n_estimators=300, random_state=42)
2
3 rf.fit(X_train,y_train)
```

```
+ RandomForestClassifier
RandomForestClassifier(n_estimators=300, random_state=42)
```

```
1 rf_prob=rf.predict_proba(X_test)[:1]
```

```
1 rf_prob=rf.predict_proba(X_test)[:1]
2 print(roc_auc_score(y_test,rf_prob))
```

0.848966518318239

Random Forest Interpretation

ROC-AUC improved substantially compared with Logistic Regression.

This indicates that nonlinear relationships and feature interactions play an important role in predicting borrower risk.

Random Forest provides stronger prediction performance while retaining some interpretability through feature importance analysis.

XGBoost

XGBoost is a leading algorithm in credit risk competitions and real-world risk modelling.

```
1 from xgboost import XGBClassifier
2 xgb=XGBClassifier(n_estimators=300,max_depth=5,learning_rate=0.05,random_State=42)
3
4 xgb.fit(X_train,y_train)
```

```
+ XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=True, eval_metric=None, feature_types=None,
               feature_weights=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.05, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=5,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=300,
               n_jobs=None, num_parallel_tree=None, ...)
```

```
1 xgb_prob=xgb.predict_proba(X_test)[:1]
2
```

```
1 print(roc_auc_score(y_test,xgb_prob))
```

0.869197376268206

XGBoost Interpretation

ROC-AUC = 0.869

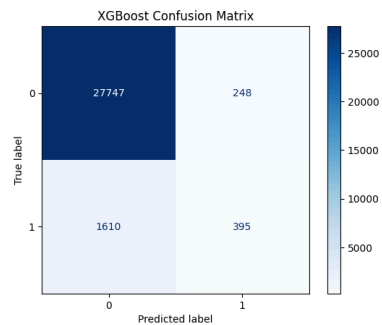
ROC-AUC measures the model's ability to distinguish between default and non-default borrowers.

Reference:

- 0.50 = Random Guessing
- 0.70 = Acceptable
- 0.80 = Strong
- Above 0.85 = Excellent

The result indicates strong discriminatory power and suggests that XGBoost is the preferred model for deployment.

```
1 from sklearn.metrics import ConfusionMatrixDisplay
2
3 ConfusionMatrixDisplay.from_estimator(
4     xgb,
5     X_test,
6     y_test,
7     cmap="Blues"
8 )
9
10 plt.title("XGBoost Confusion Matrix")
11 plt.show()
```



Confusion Matrix Interpretation

The confusion matrix helps evaluate:

- True Positives
- True Negatives
- False Positives
- False Negatives

From a lending perspective, False Negatives are generally more costly.

A False Negative occurs when a risky borrower is incorrectly classified as low risk and receives credit approval.

Model Comparison

Compare all candidate models.

```
1 results=pd.DataFrame({
2     "Model": [
3         "Logistic Regression",
4         "Random Forest",
5         "XGBoost"
6     ],
7     "ROC_AUC": [
8         roc_auc_score(y_test,lr_prob),
9         roc_auc_score(y_test,rf_prob),
10        roc_auc_score(y_test,xgb_prob)
11    ]
12 })
13 })
14
15 results.sort_values(by="ROC_AUC", ascending=False)
16
```

	Model	ROC_AUC
2	XGBoost	0.869197
1	Random Forest	0.848967
0	Logistic Regression	0.715568

Model Selection Rationale

Model comparison indicates that XGBoost achieved the highest predictive performance.

Although Logistic Regression offers superior transparency, XGBoost provides substantially stronger risk discrimination.

To balance accuracy and explainability, SHAP analysis was incorporated.

This combination is commonly used in modern credit risk analytics.

Feature Importance

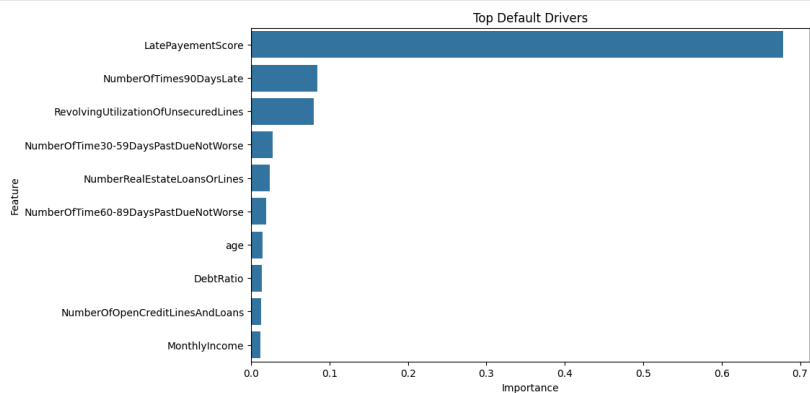
Identify the key drivers of customer default

```
1 importance=pd.DataFrame({
2     "Feature":X.columns,
3     "Importance":xgb.feature_importances_
4 })
5
6 importance=importance.sort_values(by="Importance",
7                                 ascending=False)
8
9 importance.head(10)
```

	Feature	Importance	
12	LatePaymentScore	0.678622	
6	NumberOfTimes90DaysLate	0.084152	
0	RevolvingUtilizationOfUnsecuredLines	0.080086	
2	NumberOfTime30-59DaysPastDueNotWorse	0.027713	
7	NumberRealEstateLoansOrLines	0.023319	
8	NumberOfTime60-89DaysPastDueNotWorse	0.019301	
1	age	0.014437	
3	DebtRatio	0.013995	
5	NumberOfOpenCreditLinesAndLoans	0.012596	
4	MonthlyIncome	0.012099	

Next steps: [Generate code with importance](#) [New interactive sheet](#)

```
1 plt.figure(figsize=(10,6))
2
3 sns.barplot(data=importance.head(10),
4             x="Importance",
5             y="Feature"
6 )
7
8 plt.title("Top Default Drivers")
9
10 plt.show()
```



Business Interpretation of Default Drivers

Top risk indicators include:

- LatePaymentScore
- NumberOfTimes90DaysLate
- Credit Utilisation
- Debt Ratio

These findings align with established credit risk theory.

Customers demonstrating repeated delinquency behaviour exhibit significantly higher default probability.

Explainable AI (SHAP)

Financial institutions require explainable models.

SHAP helps understand:

-Why a prediction was made -Which variables influence risk -Model transparency

```
1 import shap
2
3 explainer=shap.TreeExplainer(xgb)
4
5 shap_values=explainer.shap_values(X_test)
```

```
1 shap.summary_plot(shap_values,X_test)
```



Explainability Insights

SHAP values explain how individual variables influence model predictions.

Key observations:

- Higher delinquency metrics increase default risk.
- Higher credit utilisation increases default risk.
- Certain income and debt characteristics contribute to elevated borrower risk.

These explanations improve model transparency and support analyst decision-making.

Business Risk Assessment

In credit risk modelling, prediction errors have different costs.

False Positive

A creditworthy borrower is rejected.

Potential impact:

- Lost revenue
- Reduced customer acquisition
- Competitive disadvantage

False Negative

A risky borrower is approved.

Potential impact:

- Loan losses
- Increased default rates
- Higher capital provisioning requirements

For most lenders, False Negatives represent the greater business risk.

Executive Summary

Objective

Develop a predictive framework to identify customers likely to become seriously delinquent within two years.

Best Performing Model

XGBoost

ROC-AUC: 0.869

Key Findings

- Historical delinquency is the strongest predictor of future default.
- Credit utilisation significantly influences borrower risk.
- Debt-related characteristics contribute to elevated default probability.
- Engineered behavioural risk features improved predictive performance.

Business Recommendations

- 1. Deploy XGBoost as the primary risk-screening model.
- 2. Introduce automated alerts for customers with repeated delinquency events.
- 3. Integrate SHAP explanations into analyst review workflows.
- 4. Continuously monitor model performance and recalibrate quarterly.

Business Value

The proposed solution enables proactive identification of high-risk borrowers, improved lending decisions, and enhanced portfolio risk management.

Double-click (or enter) to edit

Double-click (or enter) to edit